



Boletín 03 Intermedio (Resuelto) — Soluciones Comentadas

Soluciones completas, comentadas y optimizadas de los 12 ejercicios de nivel intermedio de la Unidad 3 en Java 25 LTS.

A continuación se detallan las soluciones completas a los 12 ejercicios de nivel intermedio, haciendo hincapié en la modularidad limpia, la optimización algorítmica y el manejo de excepciones.

Solución Ejercicio 1: Matriz Transpuesta

```
import java.util.Arrays;

public class Ejercicio01Transpuesta {

    public static int[][] transponer(int[][] matriz) {
        if (matriz == null || matriz.length == 0 || matriz[0] == null) {
            throw new IllegalArgumentException("La matriz no puede ser nula ni vacía.");
        }

        int filasOriginales = matriz.length;
        int columnasOriginales = matriz[0].length;

        // La matriz transpuesta invierte las dimensiones: C x F
        int[][] transpuesta = new int[columnasOriginales][filasOriginales];

        for (int i = 0; i < filasOriginales; i++) {
            for (int j = 0; j < columnasOriginales; j++) {
                transpuesta[j][i] = matriz[i][j];
            }
        }

        return transpuesta;
    }

    public static void main(String[] args) {
        int[][] original = {
            {1, 2, 3},
            {4, 5, 6}
        };

        int[][] resultado = transponer(original);

        System.out.println("Original: " + Arrays.deepToString(original));
        System.out.println("Transpuesta: " + Arrays.deepToString(resultado));
    }
}
```

Solución Ejercicio 2: Suma por Filas y Columnas en Tablero 2D

```
import java.util.concurrent.ThreadLocalRandom;

public class Ejercicio02SumaMatriz {
    public static void main(String[] args) {
        int filas = 4;
        int cols = 5;
        int[][] tabla = new int[filas][cols];

        // Rellenar con números aleatorios [10, 99]
        for (int i = 0; i < filas; i++) {
            for (int j = 0; j < cols; j++) {
                tabla[i][j] = ThreadLocalRandom.current().nextInt(10, 100);
            }
        }

        int[] sumaColumnas = new int[cols];
        int sumaTotal = 0;

        System.out.println("===== TABLERO =====");
        for (int i = 0; i < filas; i++) {
            int sumaFila = 0;
            for (int j = 0; j < cols; j++) {
                System.out.printf("%4d ", tabla[i][j]);
                sumaFila += tabla[i][j];
                sumaColumnas[j] += tabla[i][j];
            }
            sumaTotal += sumaFila;
            System.out.printf("| %5d%n", sumaFila);
        }

        System.out.println("-----");
        for (int j = 0; j < cols; j++) {
            System.out.printf("%4d ", sumaColumnas[j]);
        }
        System.out.printf("| %5d%n", sumaTotal);
    }
}
```

Solución Ejercicio 3: Verificador de Matriz Simétrica

```

public class Ejercicio03MatrizSimetrica {

    public static boolean esSimetrica(int[][] m) {
        if (m == null || m.length == 0) return false;

        int n = m.length;
        // Debe ser estrictamente cuadrada
        for (int[] fila : m) {
            if (fila.length != n) return false;
        }

        // Solo es necesario comparar los elementos estrictamente por encima de la
        diagonal
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if (m[i][j] != m[j][i]) {
                    return false; // Diferencia detectada: finalización anticipada
                }
            }
        }
        return true;
    }

    public static void main(String[] args) {
        int[][] simetrica = {
            {1, 7, 3},
            {7, 4, -5},
            {3, -5, 6}
        };

        int[][] asimetrica = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };

        System.out.println("¿Es simétrica la primera? " + esSimetrica(simetrica)); //
true
        System.out.println("¿Es simétrica la segunda? " + esSimetrica(asimetrica)); //
false
    }
}

```

Solución Ejercicio 4: Algoritmo de Ordenación por Selección

```
import java.util.Arrays;

public class Ejercicio04SelectionSort {

    public static void ordenarPorSeleccion(int[] arr) {
        if (arr == null || arr.length ≤ 1) return;

        int n = arr.length;

        for (int i = 0; i < n - 1; i++) {
            int indiceMinimo = i;

            for (int j = i + 1; j < n; j++) {
                if (arr[j] < arr[indiceMinimo]) {
                    indiceMinimo = j;
                }
            }

            // Si el mínimo no está en la posición i actual, intercambiar
            if (indiceMinimo ≠ i) {
                int aux = arr[i];
                arr[i] = arr[indiceMinimo];
                arr[indiceMinimo] = aux;
            }

            System.out.printf("Paso %d: %s\n", (i + 1), Arrays.toString(arr));
        }
    }

    public static void main(String[] args) {
        int[] datos = {29, 10, 14, 37, 13};
        System.out.println("Array original: " + Arrays.toString(datos));
        ordenarPorSeleccion(datos);
        System.out.println("Array ordenado: " + Arrays.toString(datos));
    }
}
```

Solución Ejercicio 5: Búsqueda Binaria Recursiva

```
public class Ejercicio05BusquedaBinariaRecursiva {

    public static int busquedaBinaria(int[] arr, int objetivo, int inicio, int fin) {
        // Caso Base 1: El rango de búsqueda se ha agotado (no encontrado)
        if (inicio > fin) {
            return -1;
        }

        // Cálculo seguro de la posición media
        int medio = inicio + (fin - inicio) / 2;

        // Caso Base 2: Elemento localizado
        if (arr[medio] == objetivo) {
            return medio;
        }

        // Casos Recursivos:
        if (arr[medio] > objetivo) {
            // Buscar en la mitad izquierda
            return busquedaBinaria(arr, objetivo, inicio, medio - 1);
        } else {
            // Buscar en la mitad derecha
            return busquedaBinaria(arr, objetivo, medio + 1, fin);
        }
    }

    public static void main(String[] args) {
        int[] ordenado = {3, 8, 12, 19, 27, 34, 45, 60, 72};

        int idx1 = busquedaBinaria(ordenado, 27, 0, ordenado.length - 1);
        int idx2 = busquedaBinaria(ordenado, 99, 0, ordenado.length - 1);

        System.out.println("Posición de 27: " + idx1); // 4
        System.out.println("Posición de 99: " + idx2); // -1
    }
}
```

Solución Ejercicio 6: Cifrado César con `StringBuilder`

```
public class Ejercicio06CesarStringBuilder {

    public static String cifrarCesar(String texto, int desplazamiento) {
        if (texto == null) return null;

        StringBuilder sb = new StringBuilder(texto.length());
        int shift = desplazamiento % 26; // Normalizar a las 26 letras del alfabeto
        inglés

        for (int i = 0; i < texto.length(); i++) {
            char c = texto.charAt(i);

            if (c ≥ 'A' && c ≤ 'Z') {
                char nuevo = (char) ('A' + (c - 'A' + shift + 26) % 26);
                sb.append(nuevo);
            } else if (c ≥ 'a' && c ≤ 'z') {
                char nuevo = (char) ('a' + (c - 'a' + shift + 26) % 26);
                sb.append(nuevo);
            } else {
                sb.append(c); // Símbolos, espacios y números no se alteran
            }
        }
        return sb.toString();
    }

    public static void main(String[] args) {
        String mensaje = "¡Ataque Inminente a las 05:00!";
        String cifrado = cifrarCesar(mensaje, 3);
        String descifrado = cifrarCesar(cifrado, -3);

        System.out.println("Original:   " + mensaje);
        System.out.println("Cifrado:    " + cifrado);
        System.out.println("Descifrado: " + descifrado);
    }
}
```

Solución Ejercicio 7: Detector de Palíndromos Robusto

```
import java.text.Normalizer;

public class Ejercicio07PalindromoRobusto {

    public static boolean esPalindromo(String frase) {
        if (frase == null) return false;

        // 1. Eliminar tildes diacríticas mediante Normalizer NFD
        String normalizada = Normalizer.normalize(frase, Normalizer.Form.NFD);
        normalizada = normalizada.replaceAll("[\\p{InCombiningDiacriticalMarks}]", "");

        // 2. Pasar a minúsculas y eliminar todo lo que no sea letra alfabética
        String soloLetras = normalizada.toLowerCase().replaceAll("[^a-z]", "");

        // 3. Comprobación con dos punteros en extremos opuestos
        int izquierda = 0;
        int derecha = soloLetras.length() - 1;

        while (izquierda < derecha) {
            if (soloLetras.charAt(izquierda) != soloLetras.charAt(derecha)) {
                return false;
            }
            izquierda++;
            derecha--;
        }
        return true;
    }

    public static void main(String[] args) {
        String[] pruebas = {
            "Dábale arroz a la zorra el abad",
            "Anita lava la tina",
            "A man, a plan, a canal: Panama",
            "Java no es palindromo"
        };

        for (String p : pruebas) {
            System.out.printf("%s' → ¿Palíndromo? %b%n", p, esPalindromo(p));
        }
    }
}
```

Solución Ejercicio 8: Fusión Ordenada de Arrays (*Merge*)

```
import java.util.Arrays;

public class Ejercicio08MergeArrays {

    public static int[] fusionar(int[] a, int[] b) {
        if (a == null) a = new int[0];
        if (b == null) b = new int[0];

        int[] fusion = new int[a.length + b.length];
        int i = 0; // Puntero para a
        int j = 0; // Puntero para b
        int k = 0; // Puntero para fusion

        // Avanzar comparando el elemento menor de ambos arrays
        while (i < a.length && j < b.length) {
            if (a[i] ≤ b[j]) {
                fusion[k++] = a[i++];
            } else {
                fusion[k++] = b[j++];
            }
        }

        // Volcar elementos restantes de a (si quedan)
        while (i < a.length) {
            fusion[k++] = a[i++];
        }

        // Volcar elementos restantes de b (si quedan)
        while (j < b.length) {
            fusion[k++] = b[j++];
        }

        return fusion;
    }

    public static void main(String[] args) {
        int[] vector1 = {2, 5, 8, 12, 20};
        int[] vector2 = {1, 3, 7, 9, 15, 25, 30};

        int[] resultado = fusionar(vector1, vector2);
        System.out.println("Array 1: " + Arrays.toString(vector1));
        System.out.println("Array 2: " + Arrays.toString(vector2));
        System.out.println("Fusión Ordenada: " + Arrays.toString(resultado));
    }
}
```

Solución Ejercicio 9: Validador Completo de DNI Español

```
public class Ejercicio09ValidadorDNI {

    private static final String LETRAS_ASOCIADAS = "TRWAGMYFPDXBNJZSQVHLCKE";

    public static boolean validarDniCompleto(String dni) {
        if (dni == null) return false;

        // Limpieza de posibles espacios en blanco en extremos
        String texto = dni.trim().toUpperCase();

        // Regex: exactamente 8 dígitos y una letra mayúscula
        if (!texto.matches("^\\d{8}[A-Z]$")) {
            return false;
        }

        try {
            int numero = Integer.parseInt(texto.substring(0, 8));
            char letraEsperada = LETRAS_ASOCIADAS.charAt(numero % 23);
            char letraReal = texto.charAt(8);

            return letraReal == letraEsperada;
        } catch (NumberFormatException e) {
            return false;
        }
    }

    public static void main(String[] args) {
        String[] casos = {"12345678Z", "00000000T", "48712394A", "12345", "ABCDEFGHZ"};

        for (String c : casos) {
            System.out.printf("DNI '%s' → ¿Válido? %b\n", c, validarDniCompleto(c));
        }
    }
}
```

Solución Ejercicio 10: Triángulo de Pascal con Matriz Dentada

```
public class Ejercicio10PascalJagged {

    public static int[][] generarTrianguloPascal(int filas) {
        if (filas ≤ 0) {
            throw new IllegalArgumentException("El número de filas debe ser positivo.");
        }

        int[][] pascal = new int[filas][];

        for (int i = 0; i < filas; i++) {
            pascal[i] = new int[i + 1]; // Cada fila tiene i + 1 columnas
            pascal[i][0] = 1;           // Primer elemento siempre 1
            pascal[i][i] = 1;           // Último elemento siempre 1

            for (int j = 1; j < i; j++) {
                // Suma de los dos elementos 바로 superiores
                pascal[i][j] = pascal[i - 1][j - 1] + pascal[i - 1][j];
            }
        }
        return pascal;
    }

    public static void main(String[] args) {
        int filas = 6;
        int[][] triangulo = generarTrianguloPascal(filas);

        System.out.println("Triángulo de Pascal (" + filas + " niveles):");
        for (int i = 0; i < triangulo.length; i++) {
            // Sangría decorativa para forma piramidal
            System.out.print(" ".repeat((filas - i - 1) * 2));
            for (int valor : triangulo[i]) {
                System.out.printf("%4d", valor);
            }
            System.out.println();
        }
    }
}
```

Solución Ejercicio 11: Enmascarador de Tarjetas con Regex

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class Ejercicio11EnmascaradorTarjetas {

    // Coincide con 16 dígitos separados o no por espacios o guiones
    private static final Pattern PATRON_TARJETA =
        Pattern.compile("\\b(\\d{4})[- ]?(\\d{4})[- ]?(\\d{4})[- ]?(\\d{4})\\b");

    public static String enmascararTarjetas(String texto) {
        if (texto == null) return null;

        Matcher matcher = PATRON_TARJETA.matcher(texto);
        // Sustituir los primeros 12 dígitos por asteriscos conservando el último grupo
        ($4)
        return matcher.replaceAll("****-****-****-$4");
    }

    public static void main(String[] args) {
        String log = "Pago aceptado con tarjeta 4545-1234-5678-9999 por importe de
45.50€. " +
            "Tarjeta de reserva: 1111222233334444 verificada.";

        String seguro = enmascararTarjetas(log);
        System.out.println(seguro);
    }
}
```

Solución Ejercicio 12: Conteo Recursivo de Caminos en una Cuadrícula

```
public class Ejercicio12CaminosCuadrícula {  
  
    public static int contarCaminos(int n, int m) {  
        // Precondición defensiva  
        if (n ≤ 0 || m ≤ 0) {  
            return 0;  
        }  
  
        // Caso Base: En una cuadrícula de 1 fila o 1 columna solo existe 1 camino  
        recto  
        if (n == 1 || m == 1) {  
            return 1;  
        }  
  
        // Caso Recursivo: La suma de caminos moviéndose hacia abajo + moviéndose a la  
        derecha  
        return contarCaminos(n - 1, m) + contarCaminos(n, m - 1);  
    }  
  
    public static void main(String[] args) {  
        System.out.println("Caminos en 1x1: " + contarCaminos(1, 1)); // 1  
        System.out.println("Caminos en 2x2: " + contarCaminos(2, 2)); // 2  
        System.out.println("Caminos en 3x3: " + contarCaminos(3, 3)); // 6  
        System.out.println("Caminos en 3x7: " + contarCaminos(3, 7)); // 28  
    }  
}
```